DHANUSH MEKALA

dhanushmekala04@gmail.com +91-8500237057 linkedin.com/in/dhanushmekala04 github.com/dhanushmekala04

Summary

AI/ML Engineer with 1 year of hands-on experience through internships, building production-oriented ML and LLM applications. Skilled in NLP, predictive modeling, and scalable AI system development.

Experience

AI & ML Engineer Intern *Nov 2025 – May 2026*
PROFOLO — On-site

- Built a production-grade candidate intelligence pipeline integrating GitHub & LeetCode APIs, using **GPT-4 + LangChain** to generate structured evaluation reports with evidence-based scoring—reducing manual screening effort.
- Engineered a robust prompt system using LangChain templates and rubric-driven reasoning; created a prompt testing framework to ensure consistent, stable LLM outputs in production.
- Designed a skill taxonomy & role classification layer using **Pandas/NumPy** to normalize skill data; generated embeddings with **Qwen + Hugging Face** and indexed profiles in **Pinecone (7 namespaces)** for scalable semantic search.
- Contributed to **TalentGPT**, an LLM-powered recruiter search engine using hybrid retrieval (**Pinecone + MongoDB**)— improving candidate ranking relevance and retrieval accuracy.
- Supported deployment of scalable backend services using **FastAPI on AWS (Lambda, SQS, S3)**, enabling asynchronous processing and production readiness.

Technologies: GPT-4, Qwen, Hugging Face, LangChain, Pinecone, FastAPI, AWS (Lambda, SQS, S3), MongoDB

AI & ML Engineer Intern *Apr 2025 – Oct 2025*
WINIT SOFTWARE — On-site
Automated Order Processing & Escalation System

- Automated **500+ daily purchase orders** using an LLM-driven document pipeline (Gmail API, Mistral OCR, Gemini), achieving **95% extraction accuracy** across PDF, CSV, and email inputs.
- Built **stateful LangGraph workflows** with conditional branching, retry logic, and validation orchestration for resilient end-to-end document processing.
- Designed a **PostgreSQL validation engine** with **21+ business rules** detecting pricing, UOM, and promotional mismatches, reducing order errors by **45%**.
- Implemented **asynchronous task execution** using Celery and RabbitMQ, and developed a role-based escalation system with Twilio and ElevenLabs, reducing manual escalation effort by **60%**.
- Architected a **production-ready, fault-tolerant AI system** supporting high-volume enterprise workflows.

Technologies: Python, LangGraph, Gemini LLM, Mistral OCR, PostgreSQL, Celery, RabbitMQ, Twilio, ElevenLabs,

Core Skills

Languages: Python, SQL
AI / ML: Scikit-learn, XGBoost, Random Forest, SVM, Gradient Boosting, TensorFlow, Keras, PyTorch, CNN, RNN, LSTM, Transformers, Time Series Forecasting, NLP.
LLM & GenAI: LangChain, LangGraph, Hugging Face Transformers, GraphRAG, Fine-Tuning (LoRA/QLoRA), Prompt Engineering, RAG (Retrieval-Augmented Generation).
Databases: PostgreSQL, MySQL, Neo4j (Graph DB), ChromaDB (Vector DB), Pinecone (Vector DB), MongoDB
Backend: FastAPI, Flask, REST APIs, Streamlit, Docker, MLflow, GitHub Actions, Celery, RabbitMQ, Ollama
Cloud: AWS (EC2, Lambda, SQS, S3, Fargate)
AI APIs: OpenAI, Groq, Gemini, Mistral OCR, ElevenLabs, Twilio, GPT-4, Qwen, Weights & Biases

Education

Bachelor of Science (Mathematics, Statistics, and Computer Science)
Avanthi Degree and PG College, Hyderabad

Projects

Supply Chain Domain LLM — Fine-Tuned Phi-3 [GitHub]

- Fine-tuned **Microsoft Phi-3 (3.8B)** using **QLoRA (4-bit)** on **10K+ query–Cypher pairs**, improving **F1-score by 38%** for domain-specific reasoning.
- Reduced GPU memory usage from **24GB to 8GB** through parameter-efficient fine-tuning, enabling cost-efficient training.
- Built a curated evaluation dataset with **200+ domain queries**, achieving **87% first-query accuracy**.
- Converted the model to **GGUF** format and deployed via **Ollama**, achieving **sub-180ms inference latency**.

Tech Stack: PyTorch, Hugging Face Transformers, PEFT, QLoRA, Ollama, Weights & Biases

Graph-Based Supply Chain Risk Intelligence (GraphRAG) [GitHub]

- Engineered a **GraphRAG-powered knowledge graph** transforming **25K+ warehouse records** into a connected **Neo4j** graph for **multi-hop risk analysis**.
- Achieved **94% risk identification accuracy** while reducing dependency analysis to **seconds-level query latency**.
- Designed a **graph-driven risk scoring framework** modeling infrastructure, geographic, and operational dependencies.
- Built **FastAPI-based query services** integrating **LLM reasoning** for real-time risk assessment workflows.

Tech Stack: Python, Neo4j, GraphRAG, FastAPI, OpenAI

Agentic Customer Support Automation System [GitHub]

- Architected a **production-style multi-agent system** using **LangGraph** with **6 specialized agents** for ticket triage and automated resolution.
- Designed **prompt-driven agent workflows** achieving **91% automated response accuracy**.
- Implemented **stateful orchestration** with conditional branching, retry handling, and human-in-the-loop escalation, reducing response time to **seconds**.
- Developed **FastAPI microservices** with integrated latency tracking and workflow metrics.
- Deployed agent services on **AWS Lambda** with **SQS**-based asynchronous message queuing for scalable event-driven processing.

Tech Stack: LangGraph, LangChain, Groq LLaMA Models, FastAPI, Docker, AWS Lambda, SQS

Multi-Store Demand Forecasting System [GitHub]

- Built an end-to-end **CNN–LSTM forecasting system** predicting demand across **500+ SKUs**, achieving **16.4% MAPE** (**35% improvement over ARIMA**).
- Engineered **25+ temporal features** capturing seasonality and store-level demand dynamics.
- Developed scalable **FastAPI inference APIs** with Redis caching and an interactive **Streamlit dashboard**.
- Deployed forecasting services on **AWS Fargate** for containerized, scalable inference in production.

Tech Stack: Python, TensorFlow, PyTorch, FastAPI, Streamlit, Docker, AWS Fargate

Certifications

- Machine Learning Certificate — State University of New York (SUNY)
- Introduction to LangGraph — LangChain Academy
- Data Science Certification Program
- SQL Database Fundamentals
- Programming Using Python